Class-Conditional Latent Diffusion on FashionMNIST Using a Stable Diffusion VAE and Classifier-Free Guidance

Naga Sai Haritha Gottumukkala

Student ID: SID4 1598

DATA 266 Homework 9: Latent Diffusion and Stable Diffusion

***Abstract*—This paper builds a class-conditional latent diffusion model for FashionMNIST. Images are resized to 64×64, copied into three channels, and compressed by the pretrained Stable Diffusion VAE into 4×8×8 latents. This cuts the number of values per image from 12,288 to 256. A custom U-Net with 5.28 million parameters learns to predict Gaussian noise over 1,000 diffusion steps. It uses class and timestep embeddings, residual blocks, self-attention, and label dropout for classifier-free guidance (CFG). Over 50 epochs, training MSE falls from 0.5716 to 0.1163 and validation MSE falls from 0.3255 to 0.1118. Sampling with 1,000 DDPM steps and a guidance weight of 3 produces 64 labeled images. Trousers, bags, sneakers, and ankle boots look best. The main problems are blurry edges, a faint color tint, confusion between upper-body classes, no quantitative score for generated images, and the use of the official test split for validation. The network also has no cross-attention and no 16×16 attention stage, which the assignment text asked for.**

***Index Terms*—Latent diffusion, Stable Diffusion, variational autoencoder, classifier-free guidance, conditional U-Net, FashionMNIST.**

I. Introduction

Diffusion models learn to generate data by reversing a slow process that adds Gaussian noise. Running this process directly on pixels is costly, because the denoising network must handle large feature maps at every step. Latent diffusion avoids this by working in a smaller space made by an autoencoder [1]. A variational autoencoder (VAE) [5] compresses each image, a conditional U-Net removes noise in that compressed space, and the VAE decoder turns the final latent back into an image.

This work applies that idea to FashionMNIST [2], a set of ten clothing classes in 28×28 grayscale images. We pass the images through a Stable Diffusion VAE, train a small conditional denoiser, and use classifier-free guidance (CFG) [3] to generate the class we ask for. The report covers both the numbers and the saved figures. It also states clearly where the implementation differs from the assignment wording: it uses additive class conditioning instead of cross-attention, and its attention layers work at 8×8 and 2×2 rather than 16×16.

The objectives are to:

1) encode and reconstruct FashionMNIST with a pretrained VAE, using the correct normalization and latent scaling;
2) implement and visualize a 1,000-step forward diffusion process in latent space;
3) build a class-conditional latent U-Net with timestep embeddings, residual paths, self-attention, and a null class for CFG;
4) train the network for 50 epochs to predict the injected noise; and
5) sample 64 class-conditional images with the required CFG equation and the full 1,000-step DDPM reverse process [4].

II. Dataset and Experimental Setup

A. Dataset and Preprocessing

FashionMNIST has 60,000 training images and 10,000 official test images, spread evenly over ten clothing classes. Each image goes through the same fixed steps: resize from 28×28 to 64×64, repeat the gray channel three times, convert to a tensor, and normalize each channel with mean 0.5 and standard deviation 0.5. This moves pixel values from [0, 1] to [−1, 1], which is the range the VAE expects.

The official test split serves as the validation set at every epoch. This gives a stable curve, but it leaves no untouched test set for a final check. A better protocol would take validation data from the training set and use the test split once, at the end.

TABLE I
Data and Latent Sizes

Property	Value
Training images	60,000
Validation images	10,000 (official test split)
Original image size	1×28×28
VAE input size	3×64×64
VAE latent size	4×8×8
Values per input image	12,288
Values per latent	256
Compression ratio	48:1

B. Environment

All experiments ran on Google Colab with an NVIDIA Tesla T4 GPU, using PyTorch 2.11.0, torchvision 0.26.0, diffusers 0.42.1, and transformers 5.21.0. The VAE is stabilityai/sd-vae-ft-mse [6]. Forward passes use float16 mixed precision, and the loss is computed in float32. The random seed is 1598. The notebook, its run log, and requirements.txt record these settings. The first notebook cell upgrades packages without pinned versions, so anyone repeating the work should install from requirements.txt instead.

III. Method

A. VAE Compression

The VAE encoder gives a Gaussian distribution over latents for each image x. We sample a latent z from it and multiply by the Stable Diffusion scaling factor s = 0.18215 before diffusion:

$$z_0 = s\,z, \quad q_\varphi(z \mid x) = N(\mu_\varphi(x),\ diag(\sigma_\varphi^2(x))) \tag{1}$$

Before decoding, the scale is undone: $\hat{x} = D(z_0 / s)$. The code applies both steps correctly. The VAE is frozen and runs without gradients. Working on a 4×8×8 latent is much cheaper than working on a 3×64×64 image, and this saving is the main reason to use latent diffusion.

We measured reconstruction error on 64 training images. The mean squared error (MSE) in normalized [−1, 1] space was 0.00954555 (std 0.00622775). After clipping both images to [−1, 1] and mapping them to [0, 1], it was 0.00229053 (std 0.00152609). The second number is not simply the first divided by four, because the decoder sometimes outputs values outside [−1, 1] and clipping removes them. Fig. 1 shows eight examples: the overall shapes survive, but edges and texture are smoother. The VAE was trained on natural color photos, not repeated-gray clothing images, so this domain gap may play a part. We did not run an ablation to confirm it.

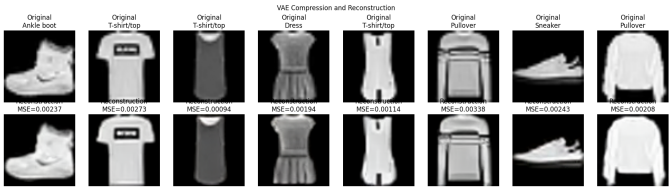


Fig. 1. VAE compression and reconstruction for eight FashionMNIST images. Top row: originals. Bottom row: reconstructions.

B. Forward Diffusion in Latent Space

The forward process uses 1,000 steps with a linear noise schedule, where β_t runs from 10^{-4} to 0.02, $\alpha_t = 1 - \beta_t$, and the cumulative product is $\bar{\alpha}_t$. A noisy latent at any step can be sampled directly from the clean latent:

$$z_t = \sqrt{\bar{\alpha}_t}\, z_0 + \sqrt{1 - \bar{\alpha}_t}\, \varepsilon, \quad \varepsilon \sim N(0, I) \tag{2}$$

The schedule array has 1,001 entries. Index 0 is a clean state with $\beta_0 = 0$ and $\bar{\alpha}_0 = 1$, and indices 1 to 1,000 are the real steps. Training and sampling both use this convention, so there is no off-by-one error. Many libraries store the 1,000 steps at indices 0 to 999 instead, so the two layouts should not be mixed.

TABLE II
Forward Diffusion Statistics

t	$\bar{\alpha}_t$	Signal $\sqrt{\bar{\alpha}_t}$	Noise $\sqrt{1-\bar{\alpha}_t}$	Latent MSE
0	1.000000	1.000000	0.000000	0.000000
200	0.659039	0.811812	0.583919	0.360481
500	0.078587	0.280334	0.959902	1.415777
800	0.001532	0.039142	0.999234	1.925303
1000	0.000040	0.006353	0.999980	1.996381

As t grows, the clean signal fades and noise takes over. By t = 500 only 28.0% of the signal amplitude is left, and by t = 1000 the signal coefficient is about 0.00635. Fig. 2 decodes these noisy latents. Because heavily noised latents lie far outside what the VAE saw in training, the late images show lost information and should not be read as natural intermediate pictures.

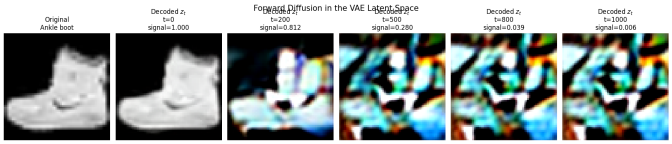


Fig. 2. Forward diffusion in latent space at t = 0, 200, 500, 800, and 1000.

C. Conditional U-Net

The denoiser takes a noisy latent of shape [B, 4, 8, 8] and returns a noise prediction of the same shape. It has 5,275,716 trainable parameters. Table III lists its parts.

TABLE III
U-Net Architecture

Component	Implementation
Base channels	64
Channel multipliers	(1, 2, 4), giving 64, 128, 256
Spatial path	8×8 → 4×4 → 2×2 → 4×4 → 8×8
Time conditioning	64-D sinusoidal embedding, MLP to 256-D
Class conditioning	11×256 embedding table (labels 0–9 plus null label 10)
Conditioning injection	Time and class vectors added, then projected into each residual block
Residual block	GroupNorm, SiLU, conv; embedding projection; GroupNorm, SiLU, conv
Downsampling	3×3 stride-2 convolution
Upsampling	Nearest-neighbor ×2, then 3×3 convolution
Skip connections	Concatenation at 4×4 and 8×8 decoder stages
Self-attention	Four heads at the 8×8 encoder and decoder stages
Bottleneck attention	Four-head self-attention at 2×2
Output	Four-channel predicted Gaussian noise

Class conditioning is additive. A learned label vector is added to the timestep vector, and the sum is fed into the residual blocks. The network also uses self-attention, where the latent positions attend to each other. It does not use cross-attention, where latent positions would query a separate conditioning signal. These three ideas are related but not the same.

The assignment also asks for attention at 16×16 and at the bottleneck. A 64×64 input gives only an 8×8 latent, so no 16×16 stage exists in this network. We placed attention at 8×8 and 2×2 instead. This fits the chosen latent size, but it does not meet the 16×16 wording literally.

D. Training

At each update we take a scaled clean latent z_0, draw a step t uniformly from 1 to 1,000, and draw noise ε. Equation (2) gives z_t. The network then minimizes the noise-prediction error:

$$L_{simple} = E[\ \|\varepsilon - \varepsilon_\theta(z_t, t, \tilde{y})\|_2^2\ ] \tag{3}$$

Here $\tilde{y}$ is the null label 10 with probability 0.1 and the true class otherwise. These null-label examples teach the same network to make an unconditional prediction, which CFG needs.

TABLE IV
TRAINING SETTINGS

Setting	Value
Epochs	50
Batch size	256
Optimizer	Adam
Learning rate	2×10^{-5}
Loss	Gaussian noise-prediction MSE
Label-drop probability	0.1 (observed at the end: 0.0992)
Timestep sampling	Uniform integers 1 to 1,000 per example
Precision	float16 autocast, float32 loss, gradient scaling
Checkpoints	Every 5 epochs, plus latest and final
Training time	7.43 minutes on a Tesla T4

Validation uses fixed timesteps and fixed noise on all 10,000 test images, so the curve is repeatable. It uses no label dropout, which makes it a little easier than the training objective.

E. Classifier-Free Guidance and Sampling

Sampling starts from z_{1000} drawn from a standard normal distribution. At every reverse step the network predicts noise twice, once with the class label and once with the null label. The assignment's guidance rule is:

$$\varepsilon_{guided} = (1 + w)\,\varepsilon_{cond} - w\,\varepsilon_{uncond}, \quad w = 3 \tag{4}$$

In the common form $\varepsilon_{uncond} + s(\varepsilon_{cond} - \varepsilon_{uncond})$, this equals $s = 4$. The code follows Equation (4) exactly. The DDPM reverse mean and variance are:

$$\mu_\theta(z_t, t) = (1/\sqrt{\alpha_t})\,(z_t - \beta_t\,\varepsilon_{guided} / \sqrt{(1 - \bar{\alpha}_t)}) \tag{5}$$

$$\tilde{\beta}_t = \beta_t\,(1 - \bar{\alpha}_{t-1}) / (1 - \bar{\alpha}_t) \tag{6}$$

For $t > 1$ the sampler draws $z_{t-1} = \mu_\theta + \sqrt{\tilde{\beta}_t}\,\eta$ with $\eta \sim N(0, I)$. At the last step it uses the mean with no added noise.

TABLE V
SAMPLING SETTINGS

Setting	Value
Samples	64
Requested labels	0 to 9, repeated (classes 0–3 seven times, classes 4–9 six times)
Guidance weight	3.0 under Equation (4)
Reverse steps	1,000
Sampling seed	1598
Latent shape	64×4×8×8 (start and end)
Final latent mean, std	−0.04307145, 1.10899949
Decoded shape	64×3×64×64
Raw decoder range	−1.48952031 to 1.16793740

IV. RESULTS

A. Training History

Loss drops quickly in the first ten epochs and then improves slowly (Fig. 3, Table VI). Training loss shows small ups and downs late in training, while validation loss keeps falling to epoch 50. The best validation value is therefore the last one, 0.11176687. The curves give no sign of overfitting. Validation stays slightly below training because it uses full labels and fixed noise, while training uses 10% label dropout and fresh noise each time. This fits the protocol but was not tested in an ablation. Loss is still falling a little at epoch 50, so full convergence is not shown.

TABLE VI
TRAINING AND VALIDATION MSE AT SELECTED EPOCHS

Epoch	Train MSE	Validation MSE
1	0.57163303	0.32552291
5	0.16927544	0.15797712
10	0.14310074	0.13580373
20	0.12689094	0.12236384
30	0.12152570	0.11696666
40	0.11776510	0.11399839
50	0.11632118	0.11176687

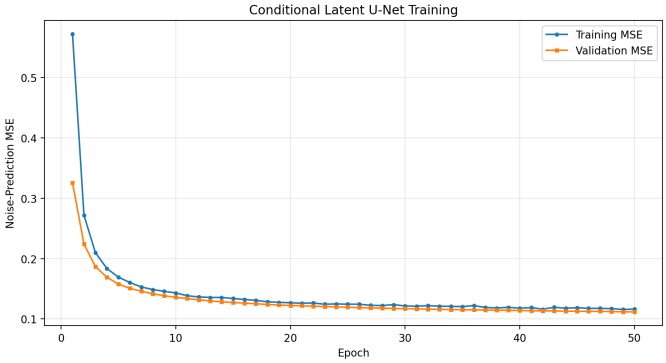


Fig. 3. Learning curves for the conditional latent U-Net over 50 epochs.

B. Reverse Diffusion

Fig. 4 decodes the latent at several points along the reverse path. Early on, the decoded images are colorful and unstructured, because noisy latents lie outside what the VAE has seen. The outline of the garment becomes clear late in the process. At step 0, most rows show a recognizable shape. The sandal row is still fragmented, and some upper-body classes look alike.

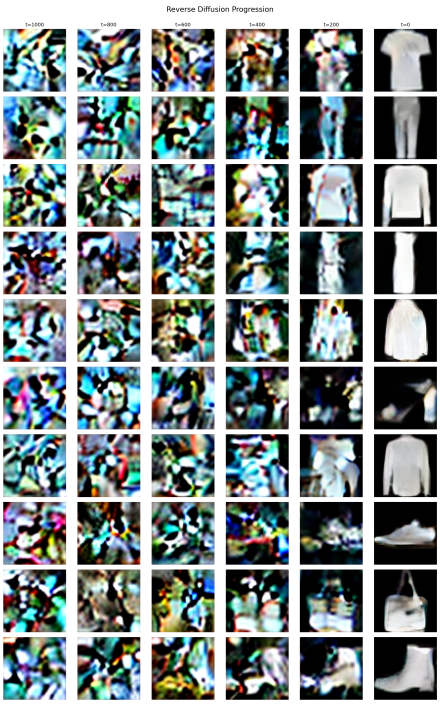


Fig. 4. Reverse diffusion for the first ten requested labels (columns: t = 1000, 800, 600, 400, 200, 0).

C. Quantitative Evaluation

Table VII collects the measured numbers. They show that the U-Net learned the noise-prediction task. They do not measure image quality or class accuracy directly, so the 64 generated images are judged by eye.

TABLE VII
SUMMARY OF MEASURED RESULTS

Target	Metric	Result	Scope
VAE fidelity	Mean MSE, normalized [−1, 1]	0.00954555	First 64 training images
VAE fidelity	Mean MSE, clipped [0, 1]	0.00229053	First 64 training images
Denoising	Training MSE, epoch 1 → 50	0.57163303 → 0.11632118	Training set
Denoising	Validation MSE, epoch 1 → 50	0.32552291 → 0.11176687	Fixed, test split
Generation	Classifier accuracy	Not measured	No claim made
Generation	FID / KID	Not measured	No claim made
Generation	Diversity score	Not measured	No claim made

D. Qualitative Evaluation

Fig. 5 shows the 64 final samples. The grid is good evidence that the class label has an effect. Trousers are usually two separate legs, bags often have handles, and sneakers and ankle boots often show a clear side view. Repeated labels differ in width, pose, outline, brightness, and tint, so we see no sign of mode collapse in this sample.

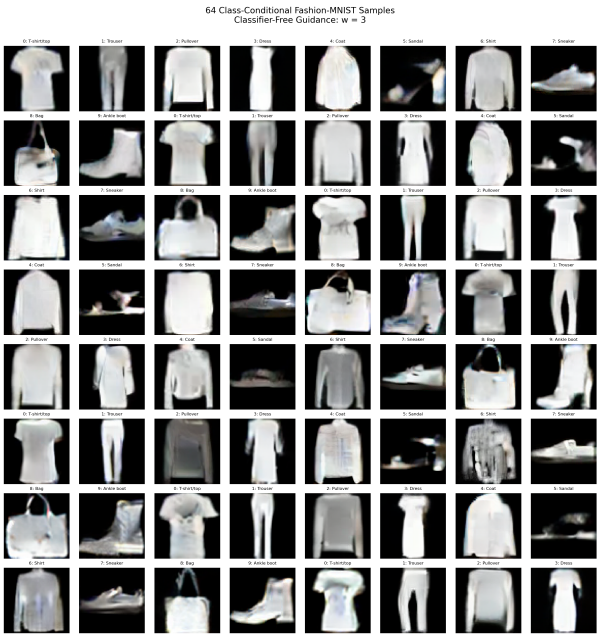


Fig. 5. 64 class-conditional samples with CFG weight w = 3. Each panel is labeled with its requested class.

Quality is uneven. Several sandals are broken into pieces. At least one dress request looks like a low, wide object. T-shirt, pullover, coat, and shirt often share almost the same torso shape. Many edges are blurry, and a pale color tint remains even though the source images are gray. Label match is therefore strongest at the level of broad shapes and weaker at the boundaries between similar classes. We saved no classifier evaluation, so we do not report an accuracy figure.

V. LIMITATIONS AND REQUIREMENT COMPLIANCE

A. Limitations

VAE and data domain. The 48:1 compression is efficient but throws away detail. Enlarging 28×28 images to 64×64 before compressing cannot bring back real fine detail. The VAE was also trained on natural color images, not gray clothing images. We see non-zero reconstruction error, smoother edges, a color tint, and decoder values outside the valid range. A VAE domain gap is a likely cause, but no other VAE was tested, so it is not proven.

Conditioning. Distinct classes are usually easy to recognize, but similar upper-body classes overlap and some sandals and dresses fail. Adding one global label vector may give weaker spatial control than cross-attention. The 10% label dropout also trades some conditional training for the unconditional branch. These are plausible reasons, but only a controlled study could separate their effects.

Sampling. Full DDPM sampling makes 1,000 predictions in a row, so small errors can build up, especially near the end when edges form. The saved progression shows broken intermediate shapes and a few malformed final objects. The colorful early decodes are not model failures by themselves, since noisy latents are outside the VAE's normal input range.

Evaluation. Only 64 samples were inspected, and we have no classifier accuracy, confusion matrix, FID, KID, or diversity

score. Claims about the full output distribution must stay qualitative. This is a gap in evaluation, not evidence that conditioning failed. Because the test split was used for validation, the evidence about generalization is also weaker than it would be with a separate final test.

B. Requirement Compliance

We checked 67 requirements against the saved code, outputs, logs, and figures, without retraining or resampling. Table VIII gives the result: 62 pass, 2 are partial, 2 fail, and 1 could not be verified.

TABLE VIII
REQUIREMENT CHECK BY AREA

Area	Pass	Partial	Fail	Unver.	Main issue
VAE compression	12	0	0	0	Clipped display MSE; 64 images
Forward diffusion	8	0	0	0	1-based step convention, documented
Conditional U-Net	14	0	2	0	No cross-attention; no 16×16 attention
Training	13	1	0	0	Test split used as validation
CFG and sampling	15	0	0	0	Quality judged by eye only
Cross-cutting	0	1	0	1	Notebook placeholders; TA notebooks missing
Total	**62**	**2**	**2**	**1**	**67 requirements**

The two failures are literal wording issues in the architecture. Fixing either one would change the network and need retraining, so we did not change them. The partial item in the last row is notebook cell 19, which still has two [INSERT VALUE] markers. The correct numbers, 0.00229053 and 0.00954555, appear in cell 14. The unverified item is the comparison with the TA demonstrations. The files Demo_9_CFG.ipynb and Demo_9_Stable_Diffusion (1).ipynb were not available, so we compared against the written requirements only. The implementation meets the VAE, diffusion, training, and sampling requirements, and it adapts the attention resolution and conditioning method as described in Section III-C.

VI. FUTURE WORK

The results point to several concrete next steps:

1) Ask the instructor whether literal cross-attention and 16×16 attention are required for grading. If they are, redesign and retrain the network instead of relabeling the current layers.
2) Split the 60,000 training images into training and validation sets, and keep the official test split for one final evaluation.
3) Measure VAE reconstruction error on the full evaluation set, and compare posterior samples with posterior means.
4) Train a separate FashionMNIST classifier to measure class accuracy and confusion in generated images. Add FID or KID and a diversity measure.
5) Compare several guidance weights from the same starting noise to measure the trade-off between image quality and variety.
6) Try a FashionMNIST-specific autoencoder or a larger latent grid to reduce blur and color tint.
7) Test EMA weights, timestep-aware loss weighting, or a faster sampler, while keeping the required baseline.
8) Install pinned versions from requirements.txt before running the notebook, and replace the two [INSERT VALUE] markers in cell 19.

VII. CONCLUSION

This work runs the full latent diffusion pipeline on FashionMNIST: VAE compression, Gaussian noising, conditional noise prediction, classifier-free training, 1,000-step DDPM sampling, and VAE decoding. Training was stable, and the final training and validation MSE were 0.1163 and 0.1118. The 64 generated images show varied, recognizable class-conditioned shapes, but also blurry edges, color tint, confusion between upper-body classes, and some malformed samples.

The logs, checkpoints, latents, and saved figures support the implementation well. It does not follow the strict architecture wording, because additive conditioning is not cross-attention and the latent network never reaches 16×16 resolution. These differences should be stated openly when the work is submitted. If the instructor treats them as mandatory, the current checkpoint cannot meet them without a redesign and retraining.

REFERENCES

[1] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-resolution image synthesis with latent diffusion models," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2022. [Online]. Available: https://arxiv.org/abs/2112.10752

[2] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-MNIST: A novel image dataset for benchmarking machine learning algorithms," 2017, arXiv:1708.07747. [Online]. Available: https://arxiv.org/abs/1708.07747

[3] J. Ho and T. Salimans, "Classifier-free diffusion guidance," 2022, arXiv:2207.12598. [Online]. Available: https://arxiv.org/abs/2207.12598

[4] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2020. [Online]. Available: https://arxiv.org/abs/2006.11239

[5] D. P. Kingma and M. Welling, "Auto-encoding variational Bayes," 2013, arXiv:1312.6114. [Online]. Available: https://arxiv.org/abs/1312.6114

[6] Hugging Face, "stabilityai/sd-vae-ft-mse," model repository. [Online]. Available: https://huggingface.co/stabilityai/sd-vae-ft-mse